Advanced protocol engineering and security lab

EXP-4,5,6

Name: **Poloju Varshith**

Roll no: ch.sc.u4cys23035

EXPERIMENT 4

AIM: To study the effect of queuing delay and buffer overflow in a network using UDP traffic and observe packet loss under congestion conditions**.**

DESCRIPTION: In this experiment, UDP traffic is generated across a bottleneck link. Since UDP is a non-adaptive protocol, it does not adjust its sending rate during congestion. As traffic increases, packets get queued in buffers, causing queuing delay. When the buffer limit is exceeded, packet loss occurs due to buffer overflow**.**

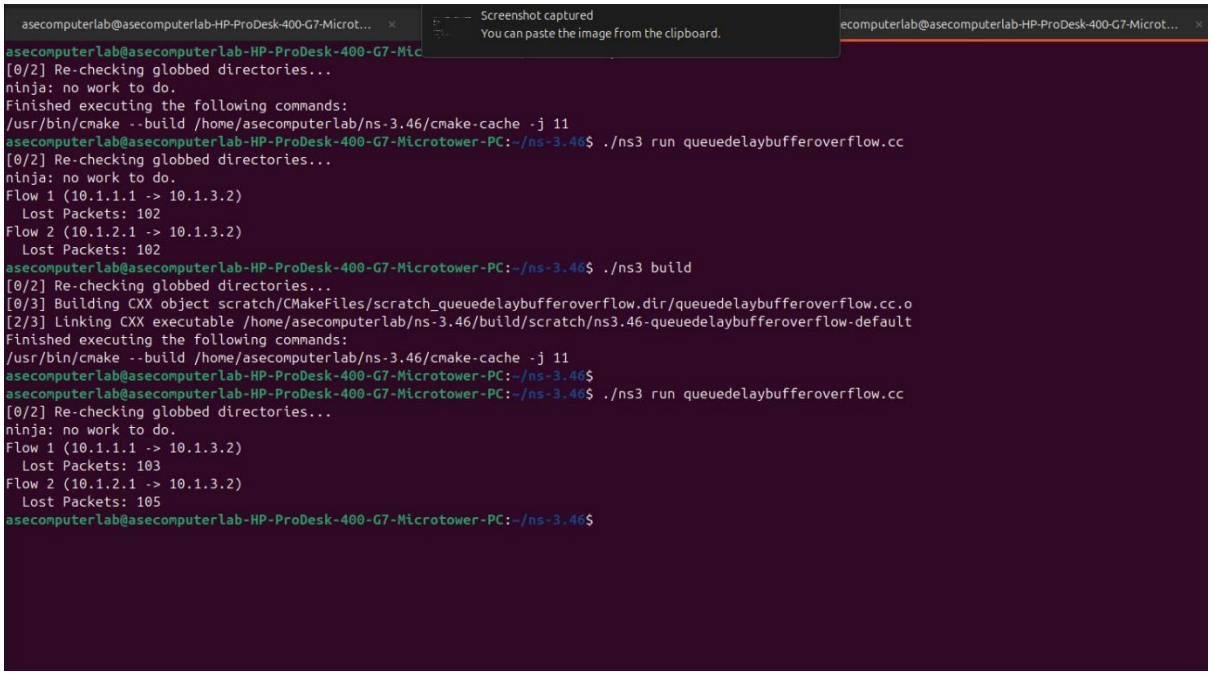
PROGRAM USED: queuedelaybufferoverflow.cc

Program code:

server.py | point-to-pointdelay.cc | MultiHopDelayAnalysis.cc | • queuedelaybufferoverflow.cc

```
int main(int argc, char *argv[]) {
    CommandLine cmd;
    cmd.Parse(argc, argv);

    NodeContainer clients, router, server;
    clients.Create(2);
    router.Create(1);
    server.Create(1);

    // Access links: 100 Mbps
    PointToPointHelper accessLink;
    accessLink.SetDeviceAttribute("DataRate", StringValue("100Mbps"));
    accessLink.SetChannelAttribute("Delay", StringValue("2ms"));

    // Bottleneck: 5 Mbps, tiny 5-packet queue
    PointToPointHelper bottleneck;
    bottleneck.SetDeviceAttribute("DataRate", StringValue("25Mbps"));
    bottleneck.SetChannelAttribute("Delay", StringValue("10ms"));
    bottleneck.SetQueue("ns3::DropTailQueue<Packet>",
                        "MaxSize", QueueSizeValue(QueueSize(QueueSizeUnit::PACKETS, 5)));

    NetDeviceContainer d02 = accessLink.Install(clients.Get(0), router.Get(0));
    NetDeviceContainer d12 = accessLink.Install(clients.Get(1), router.Get(0));
    NetDeviceContainer d23 = bottleneck.Install(router.Get(0), server.Get(0));

    InternetStackHelper stack;
    stack.Install(clients);
```

Execution:



Result:

Thus, the behavior of queuing delay and buffer overflow in UDP traffic was successfully studied

EXPERIMENT 5

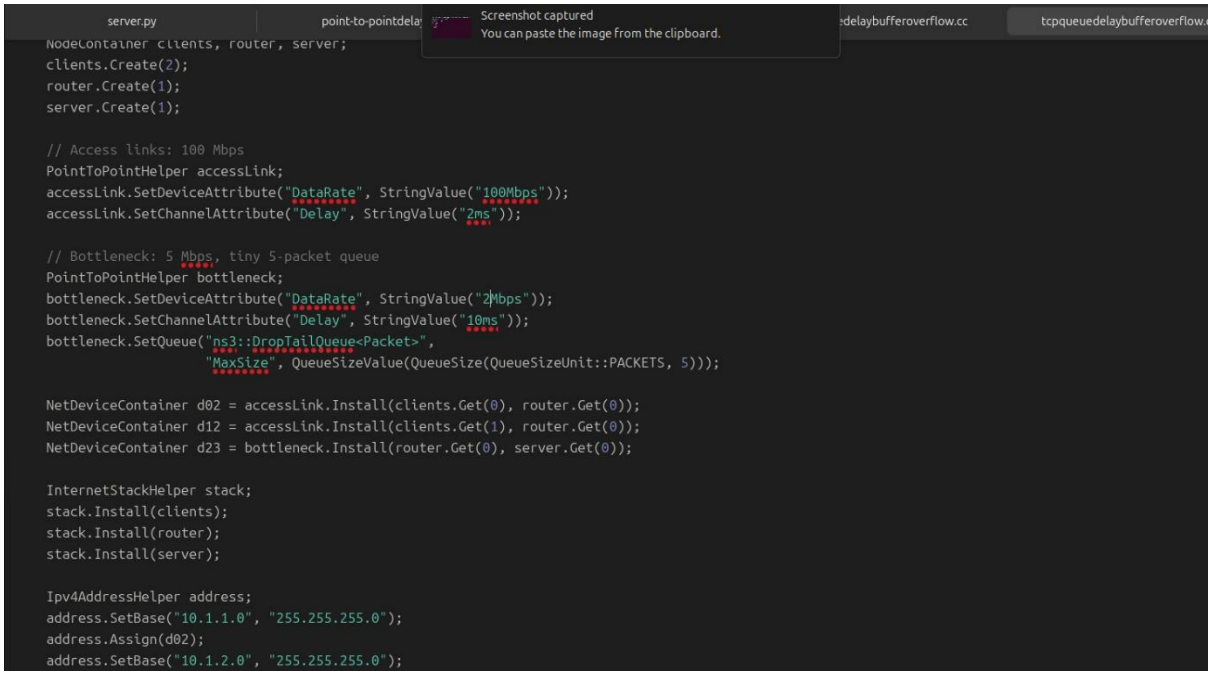
QUEING DELAY AND BUFFER OVERFLOW -TCP

AIM: To analyze queuing delay and buffer overflow in a network using TCP traffic and observe congestion control behavior.

DESCRIPTION: In this experiment, TCP traffic is transmitted across a shared bottleneck link. TCP is an adaptive protocol that dynamically adjusts its congestion window based on network conditions. When congestion occurs, TCP reduces its sending rate to minimize packet loss.

tcpqueuedelaybufferoverflow.cc

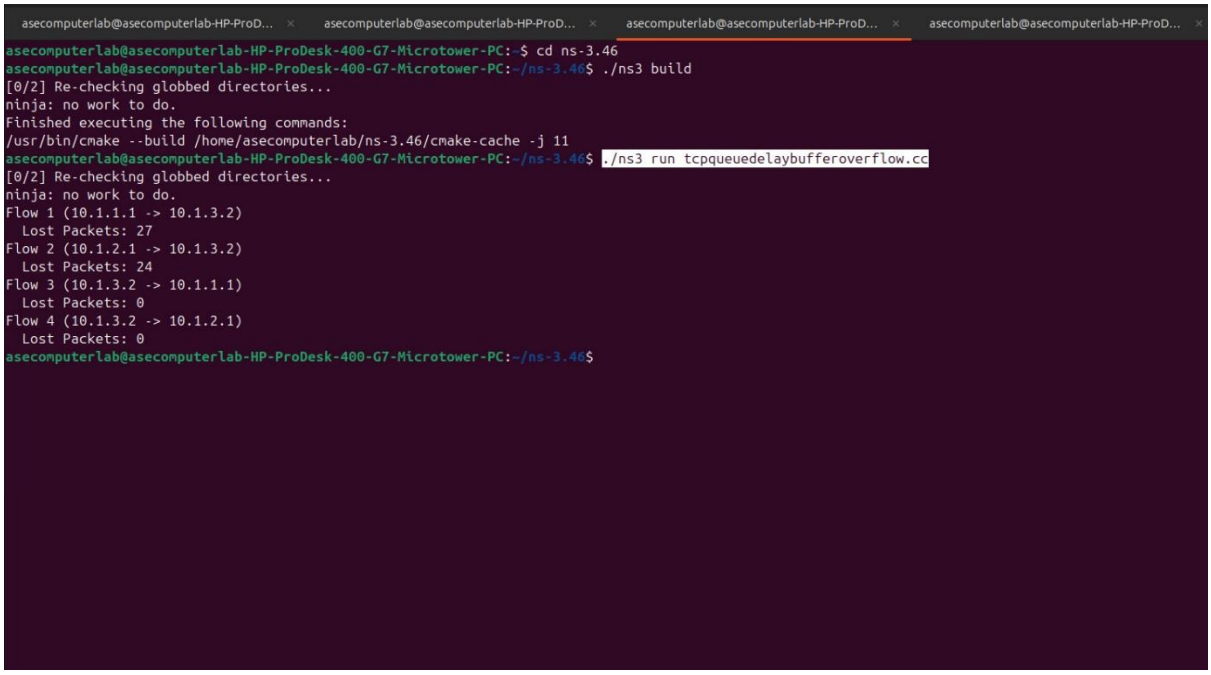
Program code:

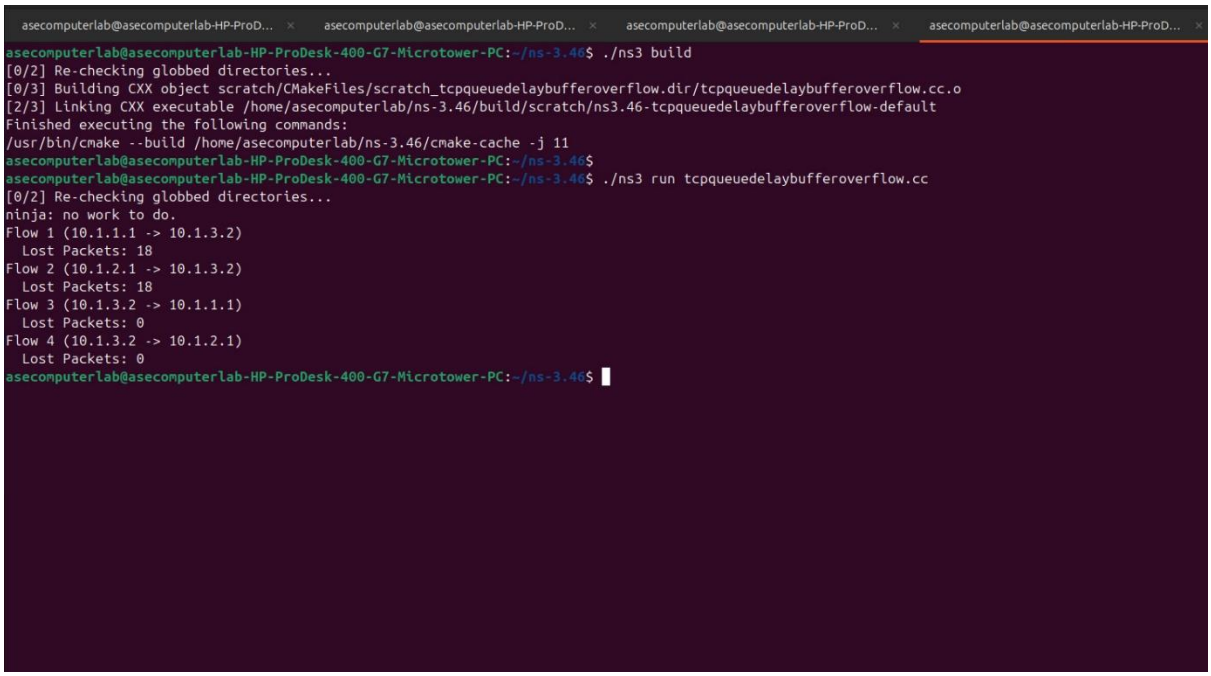


Execution:

1st = Bottleneck link: 5 Mbps

2nd = Bottleneck link: 2 Mbps





Result:

Thus, queuing delay and buffer overflow behavior in TCP traffic was successfully analyzed.

Experiment 6

Aim: To study and compare the behavior of adaptive TCP traffic and non-adaptive UDP traffic over a shared bottleneck link by observing TCP congestion window dynamics, throughput, and packet loss under congestion.

Description:

This experiment compares TCP and UDP traffic when both share a congested network link. TCP adapts its transmission rate based on congestion feedback, whereas UDP continues sending packets without adaptation, leading to unfair bandwidth usage and packet loss.

tcpvsudp.cc

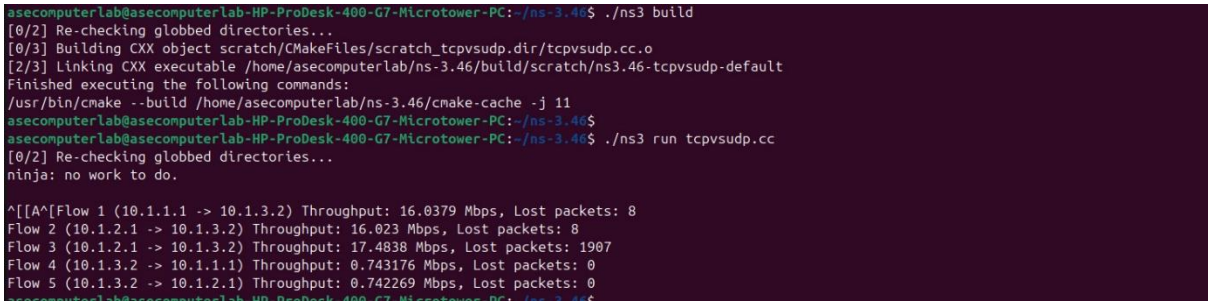
Program code:

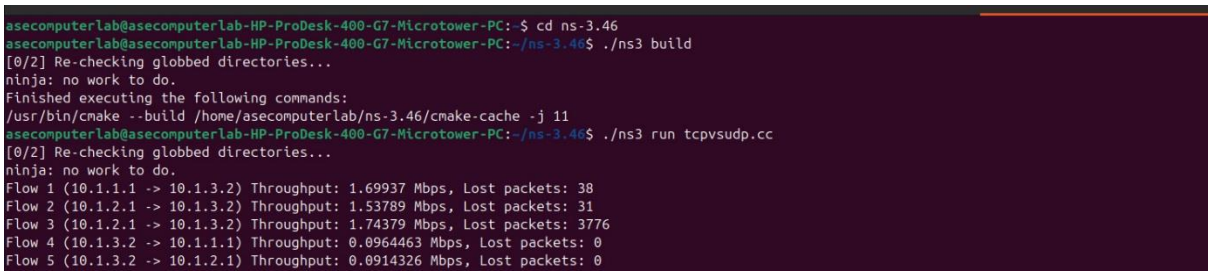
```
clients.Create(2);
router.Create(1);
server.Create(1);

// Access links: 100 Mbps
PointToPointHelper accessLink;
accessLink.SetDeviceAttribute("DataRate", StringValue("100Mbps"));
accessLink.SetChannelAttribute("Delay", StringValue("2ms"));

// Bottleneck link: 5 Mbps, 5-packet queue
PointToPointHelper bottleneck;
bottleneck.SetDeviceAttribute("DataRate", StringValue("50Mbps"));
bottleneck.SetChannelAttribute("Delay", StringValue("10ms"));
bottleneck.SetQueue("ns3::DropTailQueue<Packet>",
                    "MaxSize", QueueSizeValue(QueueSize(QueueSizeUnit::PACKETS, 5)));

NetDeviceContainer d02 = accessLink.Install(clients.Get(0), router.Get(0));
NetDeviceContainer d12 = accessLink.Install(clients.Get(1), router.Get(0));
NetDeviceContainer d23 = bottleneck.Install(router.Get(0), server.Get(0));

InternetStackHelper stack;
stack.Install(clients);
stack.Install(router);
stack.Install(server);

Ipv4AddressHelper address;
address.SetBase("10.1.1.0", "255.255.255.0");
address.Assign(d02);
address.SetBase("10.1.2.0", "255.255.255.0");
address.Assign(d12);
```

Execution:

1st = Bottleneck link: 5 Mbps

2nd = Bottleneck link: 50 Mbps





Result:

Thus, the behavior of adaptive TCP traffic and non-adaptive UDP traffic over a shared bottleneck link was successfully studied and compared.